FLIGHT SUPPORT

Your Flight Support Bot System

Developer presentation guide

Architecture, data flow, source files, functions, operations and testing

Project purpose

A source-grounded French-language flight information assistant that indexes a fixed set of public Tunisair pages, answers from retrieved evidence, and sends travelers to official pages for live or booking information.

Prepared from the repository implementation

Source review date: 11 November 2026

Stack: Python · FastAPI · LangGraph · Ollama · Pinecone · SQLite · HTML/CSS/JavaScript

This guide describes the checked-in application code and the local sync setup. Do not put credentials in slides or screenshots.

01 · ORIENTATION

Executive overview

This project is a small web application with two distinct jobs: keep a carefully selected public knowledge base refreshed, and answer user questions through a controlled retrieval-and-generation workflow.

What it does

- Fetches a fixed allowlist of public Tunisair French pages.
- Extracts readable HTML text, checks it for access challenges, and stores the full extracted text in SQLite.
- Splits the pages into overlapping text chunks, embeds them with Ollama, and upserts them into a Pinecone namespace.
- Uses LangGraph to route, retrieve, evaluate evidence, and produce a cited answer.
- Returns official links for requested pages, flight-status checks, and booking searches.

What it deliberately does not do

- It does not crawl arbitrary site links or infer permission from a URL being on the same domain.
- It does not bypass CAPTCHA, HTTP blocks, or booking-site restrictions.
- It does not claim current availability or fares from a weekly snapshot.
- It does not automate the booking transaction.
- It does not guarantee every topic on the airline website is indexed.

Approved source set in this version

1. https://www.tunisair.com/fr
2. https://www.tunisair.com/fr/offres
3. https://www.tunisair.com/fr/vols-en-cours
4. https://flights.tunisair.com/fr

> **Important:** “Detailed data” here means the complete readable text extracted from these four pages, not every linked destination or offer detail page. The fetcher does not follow internal links. Add a new URL to the fixed allowlist only after reviewing its access rules and purpose.

Observed initial synchronization: four source pages and 33 vector chunks. Those numbers can change if the source HTML changes.

02 · ARCHITECTURE

System architecture

Traveler → French web UI → FastAPI `POST /api/chat` → LangGraph workflow → (SQLite/Pinecone or official link) → response with sources and trace

Runtime components

Component	Role	Runtime connection
Browser	Flight form, assistant chat, citations and trace.	Served by FastAPI on port 8080.
FastAPI / Uvicorn	Serves the HTML page, static assets and JSON API.	`python run.py`; `127.0.0.1:8080`.
LangGraph	Stateful routing and conditional branches.	In-process workflow in `app/rag/workflow.py`.
Ollama	Local chat model and embeddings.	Docker-exposed host endpoint `http://localhost:11434`.
Pinecone	Semantic vector search and indexed chunks.	Cloud service, API key from environment.
SQLite	Full extracted source text and request audit.	`data/tunisair_content.db`, `data/audit.db`.

Two separate flows

Update flow: a CLI/weekly task fetches the allowed website pages, parses them, writes the page archive, creates chunks, embeds and upserts changed chunks, then writes a hash/vector-ID manifest.

Question flow: the API validates a question, asks the workflow to decide the path, retrieves only when the question needs indexed knowledge, then returns an answer plus citation links and trace.

> Pinecone is the search index, not the durable full-page archive. SQLite preserves extracted source text so it can be inspected and retained separately from the vectors.

03 · RAG

Question-answer workflow

1. **Normalize and classify.** Deterministic checks recognize official-link requests, time-sensitive flight-status questions, and fare/booking questions. Other questions use the Ollama structured-output router.
2. **Choose a branch.** Company-information questions use RAG; greetings use a direct response; page-link requests return the requested official URL; live status points to the current official page; booking questions validate explicit trip details.
3. **Retrieve evidence.** The Pinecone retriever returns the configured top-k chunks, currently four.
4. **Grade evidence.** The LLM returns a typed `good` or `weak` decision. The app does not treat the model's answer as proof that a source contains the fact.
5. **Bounded recovery.** Weak evidence may trigger one query rewrite and one additional retrieval. If still weak, the workflow reports insufficient evidence instead of fabricating an answer.
6. **Generate and cite.** The answer model is instructed to stay within retrieved text and include citations derived from source metadata.

Branch map

```
START → route_question
  ├─ official_page_link / live_flight_status → return_router_answer → END
  ├─ booking → create_booking_link → END
  ├─ direct → direct_answer → END
  └─ kb → retrieve_kb → grade_kb
       ├─ good → generate_from_kb → END
       └─ weak → rewrite_query → retrieve_kb (bounded by max_retries)
                  └─ weak after retry → insufficient → END
```

Why some questions are not answered from RAG

Prices, availability and live flight status change. A weekly snapshot can help explain static site content, but it is not a dependable source for current operational facts. The workflow therefore directs fare requests to the flight planner and flight-status requests to the live status page.

Structured routing and grading use Ollama function-calling output. This was selected because JSON-mode output from the local model previously failed to provide the required route field.

04 · DATA INGESTION

How website data is collected

Controlled crawl

`TunisairCrawler` checks the host's `robots.txt` for every fixed source URL, requests only the allowlist, refuses redirects, enforces a response-size limit and crawl delay, and stops on HTTP 401/403/421/429 or known CAPTCHA/challenge text. It never uses the booking host as a source.

Parsing and extraction

The standard-library HTML parser collects the title and visible text while ignoring script, style, navigation, header, footer, SVG and noscript sections. It decodes the page using the HTTP charset. The extractor is intentionally simple: it does not execute JavaScript or render the page like a browser.

Current source contents (observed during the initial run)

Page	Useful content observed	Freshness rule
Home	Travel planning and public website content.	Weekly indexed snapshot.
Offers	Offer listings and route names available in page HTML.	Weekly indexed snapshot; check source for current terms.
Flights in progress	Status page structure and flight-related fields/text.	Use the site directly for live status.
Flights search landing page	Public flight search landing content.	Booking availability itself is not crawled.

> Observed allowlist dry run returned 4 pages and 33 chunks on 11 November 2026. A web page can change, block access or present a challenge at any later run; sync reports that explicitly and must not be forced past it.

Adding a source safely

1. Confirm the exact URL is intended and public.
2. Review that host's robots rules and any terms or access requirements.
3. Add the exact URL to `ALLOWED_PAGE_URLS`; do not add a broad route wildcard.
4. Add or update a test for exact fetch behavior.
5. Run the dry-run, inspect the extracted content, then sync.

05 · PERSISTENCE

Storage and synchronization

Storage	What it contains	Why it exists
`data/tunisair_content.db`	One row per active source URL: title, full extracted text, SHA-256 hash, fetch time and sync time.	Durable local copy for inspection and source archive.
Pinecone namespace `tunisair-fr`	Chunk embeddings and source metadata.	Semantic retrieval at question time.
`data/tunisair_sync_manifest.json`	Per-page content hash and deterministic chunk IDs.	Detect unchanged pages and safely replace/remove obsolete vectors.
`data/audit.db`	UTC timestamp, raw user question, selected source and trace.	Basic operational audit; protect it like user data.

Incremental sync sequence

```
Fetch and validate every allowlisted page
        ↓
Extract title + readable text; calculate SHA-256
        ↓
Save full page records to SQLite
        ↓
Split changed pages (900 characters, 120 overlap)
        ↓
Ollama embeddings → Pinecone upsert with stable UUID5 IDs
        ↓
Delete obsolete vector IDs → atomically replace manifest
```

If pages are unchanged, Pinecone upserts are skipped. A blocked fetch happens before storage changes. If embedding/indexing fails after the SQLite archive is written, the run fails and the manifest remains unchanged; a later run can retry indexing.

Schedule and operations

- `setup_weekly_sync.ps1` registers a Windows Task Scheduler task for Sundays at 03:00.
- The current task is configured for the interactive Windows user; user sign-in and Docker/Ollama availability may be required.
- `run_weekly_sync.ps1` invokes the venv Python and appends output to `data/tunisair_sync.log`.
- The task was verified with Last Task Result 0 during setup. Check Task Scheduler and the log after each run.

> Data freshness is bounded by the weekly schedule. For a live flight status or price, the assistant directs the user to the official website rather than presenting a potentially stale indexed value.

06 · SOURCE FILES

Repository map: application core

File	Responsibility and key symbols
`app/main.py`	Creates the FastAPI app, initializes logging/audit DB, mounts API and static assets, and renders `home(request)`.
`app/api/routes.py`	Defines request validation through `ChatRequest`; `health()` returns service status; `chat(payload)` runs the graph, writes audit and shapes the JSON response.
`app/core/config.py`	`BASE_DIR`, Pydantic `Settings`, and cached `get_settings()`. Controls model, Pinecone, crawl, paths, retries and app label.
`app/core/logging.py`	`configure_logging()` configures timestamped INFO-level application logging.
`app/rag/state.py`	`RouteDecision` and `EvidenceGrade` define structured LLM outputs; `AgentState` defines fields passed between graph nodes.
`app/rag/vectorstore.py`	Ollama embeddings, Pinecone index safety, vector-store singleton, retriever, upsert and delete helpers.
`app/rag/workflow.py`	Route detection, trip parsing, graph nodes, answer generation, citations and graph compilation.
`app/services/ingestion.py`	Supported local-document loaders and shared recursive text chunking.
`app/services/audit.py`	Creates and writes the SQLite query-audit table.
`app/services/content_store.py`	`store_pages(...)` creates/updates SQLite source rows and removes sources no longer active.
`app/services/tunisair.py`	Fixed URL allowlist, respectful fetch/parser, access-block errors, and validated booking-link builder.
`app/services/tunisair_sync.py`	Hash-based incremental sync between crawler, SQLite, chunker, manifest and Pinecone.

Application entry points

- `run.py`: launches Uvicorn at `127.0.0.1:8080` with reload enabled for development.
- `sync_tunisair.py`: CLI with `--dry-run` and full sync modes; returns exit code 2 for a deliberate access stop and 1 for other sync errors.
- `app/api/__init__.py`, `app/core/__init__.py`, `app/rag/__init__.py`, `app/services/__init__.py`: package markers.

07 · FUNCTION REFERENCE

Workflow functions: routing and helpers

Symbol	What it does
`llm()`	Lazily creates and reuses the configured `ChatOllama` model.
`add_trace(state, message)`	Returns an appended copy of the decision trace for the current graph state.
`_normalize_text(text)`	Case-folds and removes Unicode diacritics for matching French accented keywords.
`_extract_flight_search(question)`	Finds known airport aliases/codes and supported explicit dates; returns route/date fields without guessing unknown city codes.
`_is_flight_search_request(question)`	Recognizes fare questions or flight topics paired with a booking/search action.
`_is_french_question(question)`	Uses a small keyword heuristic to choose French wording for fixed workflow responses.
`_requested_official_link(question)`	Maps a link request for offers, flight status, or general planning to an exact official URL.
`_is_live_flight_status_request(question)`	Identifies current flight-status questions that must not rely on weekly indexed data.
`route_question(state)`	Handles deterministic link/status/fare cases first; otherwise asks Ollama for a structured KB, booking, or direct route.
`route_after_router(state)`	Maps route labels to graph nodes: retrieval, booking, direct answer, or a prepared page-link answer.
`build_graph()`	Registers LangGraph nodes and conditional/terminal edges, then compiles the graph.
`ask(question)`	Creates a fresh initial `AgentState` and invokes the compiled graph.

Why route deterministically before using the LLM?

Official page links and rapidly changing flight status are predictable cases. Handling them with explicit rules avoids unnecessary retrieval and keeps live status linked to the source site. Fare requests are also classified as booking intent so that missing dates lead to a useful prompt instead of an unsupported “no information” answer.

Location extraction uses a maintained alias dictionary, not a worldwide airport database. Add a city/code mapping and tests before expecting reliable extraction for new airports.

08 · FUNCTION REFERENCE

Workflow functions: RAG and responses

Symbol	What it does
`retrieve_kb(state)`	Runs Pinecone semantic retrieval on the current query and records the chunk count.
`grade_kb(state)`	Builds evidence context and requests a structured `good` / `weak` grade through Ollama function calling.
`after_kb(state)`	Chooses answer generation, one query rewrite, or insufficient-evidence response using `max_retries`.
`rewrite_query(state)`	Asks the chat model to rephrase the user question without adding facts and increments retry count.
`generate_from_kb(state)`	Generates an answer constrained to retrieved page text and deduplicates citations by source URL.
`create_booking_link(state)`	When route/dates are missing, explains fare dependence and asks only for missing trip details. When complete, validates fields, builds the timetable URL and never fetches the booking host.
`direct_answer(state)`	Uses the chat model for short non-domain conversational messages.
`return_router_answer(state)`	Returns the fixed official-link or live-status response prepared by the router.
`insufficient(state)`	Returns a fixed message when bounded retrieval does not provide enough evidence.

Trip parsing and date support

Current aliases include Tunis/TUN, Djerba/DJE, Monastir/MIR, Paris/PAR, Lyon/LYS, Marseille/MRS, Barcelona/BCN, and Montréal/YUL. Supported explicit formats are ISO dates (`2026-11-20`), day/month/year numeric dates, and named French/English months. Invalid dates are ignored by extraction; final booking URLs are validated again by the service.

Structured state

`AgentState` carries the original question, current rewritten query, retrieved `Document`s, flight fields, evidence grade, answer, route/source label, retry count, trace and citations. Each graph node returns only the fields it updates; LangGraph merges those updates into state.

> Structured fields from the LLM are not trusted as a substitute for validation. Route and date extraction is rechecked before a booking URL is built; source-page links come from a fixed mapping.

09 · FUNCTION REFERENCE

Crawler, storage and index functions

`app/services/tunisair.py`

Class / method	Purpose
`TunisairAccessBlocked`	Explicit exception for a denied page or challenge; distinguishes “stop fetching” from ordinary failures.
`_NoRedirectHandler.redirect_request()`	Returns no redirect request so HTTP redirects are not automatically followed.
`TunisairPage`	Immutable record: URL, title, extracted text and fetch timestamp.
`_PageParser.__init__()`	Initializes ignored-tag depth, title collection and text collection.
`handle_starttag()`	Tracks title/ignored HTML sections and void tags.
`handle_endtag()`	Closes title and ignored-section depth tracking.
`handle_data()`	Normalizes whitespace and saves visible text outside ignored sections.
`TunisairCrawler.__init__()`	Creates per-host robots cache and per-crawler request-delay clock.
`_open(request)`	Builds a no-redirect opener and applies the configured request timeout.
`_fetch(url)`	Enforces allowlist/query rules, delay, headers, body-size limit, redirect check and access-denied handling.
`_robots_for(url)`	Fetches/parses host robots.txt once and caches it; refuses unsafe robots redirects.
`_parse(url, body, headers)`	Decodes HTML, extracts title/text, rejects challenges/empty text and timestamps the page.
`fetch_allowed_pages()`	Checks robots permission then fetches/parses each exact approved URL.
`build_booking_url(...)`	Validates three-letter codes and ISO date ordering, then encodes a round-trip timetable URL.

Sync, vector store, API and UI

File / function	Purpose
`tunisair_sync._read_manifest(path)`	Loads and validates JSON page-hash/vector-ID state.
`_write_manifest(path, manifest)`	Writes a temporary JSON file then atomically replaces the manifest.
`_chunks_for_page(page)`	Hashes page title/text, builds metadata-bearing LangChain documents, chunks text and creates stable UUID5 IDs.
`sync_tunisair()`	Fetches all pages first, archives them in SQLite, indexes changed chunks, removes stale vectors, writes manifest and returns counts.
`content_store.store_pages(...)`	Creates the SQLite table, upserts complete text/metadata, prunes removed URLs and surfaces database errors.
`vectorstore.get_embedding_dimension(name)`	Maps supported embedding names to dimensions and raises for unknown models.
`get_embeddings()`	Lazily creates Ollama's embedding client.
`ensure_index()`	Requires Pinecone credentials, validates dimension and creates a missing compatible index; does not delete a mismatched index.
`get_vectorstore()` / `get_retriever()`	Creates cached Pinecone store in configured namespace and returns a top-k retriever.
`add_documents(chunks, ids)` / `delete_documents(ids)`	Upserts or deletes vector records.
`routes.health()` / `routes.chat(payload)`	Health JSON; validated chat API, audit write, response serialization and HTTP error conversion.
`audit.init_db()` / `audit.write_audit(...)`	Creates audit schema and records each question/source/trace with a UTC timestamp.

Text chunking and local file loading

`load_file(path)` supports PDF, TXT, Markdown and DOCX. `chunk_documents(docs)` uses recursive splitting with 900-character chunks and 120-character overlap. The Tunisair sync uses the shared chunker. The local-file loader is not exposed as a user upload route in the current API.

11 · FRONT END

Interface files and JavaScript functions

HTML and CSS

- `templates/index.html` renders the French travel hero, route/date search form, suggested questions, chat, citations/trace area and official-site links.
- `static/css/style.css` defines the responsive light/red travel theme, layout, forms, chat, focus/disabled states and mobile breakpoints.
- Form control IDs are part of the JavaScript contract: `chatForm`, `question`, `flightSearchForm`, `originCode`, `destinationCode`, `departureDate`, `returnDate` and `swapRoute`.

`static/js/app.js` function reference

Function	Purpose
`escapeHtml(s)`	Escapes HTML-significant characters before text is inserted in message markup.
`formatText(s)`	Escapes text and converts newlines to line breaks.
`sourceLabel(source)`	Maps backend source IDs to readable UI labels.
`addMessage(role, text, source, citations)`	Appends a chat bubble with safe text, optional source and source links.
`renderTrace(items)`	Shows the workflow trace or empty-state text.
`askAgent(q)`	Posts JSON to `/api/chat`, manages loading state, renders answer/errors, citations and trace.
`localDateString(date)`	Formats a Date in local time as YYYY-MM-DD without UTC date drift.
`setReturnDateMinimum()`	Requires return date on/after departure and corrects an earlier selected return date.
Event handlers	Connect chat and example buttons; set default trip dates; swap route; validate distinct airports; submit flight searches as explicit assistant prompts.

> The flight form produces a search request through the chat workflow. It does not submit a booking or guarantee a fare. External official links open in a separate tab with `noopener noreferrer`.

Configuration, dependencies and adjacent files

Important settings

Setting	Current purpose / default
`app_name`	Browser/API label: Your Flight Support Bot System.
`pinecone_api_key`	Required credential to connect the vector database; set privately in environment.
`pinecone_index_name`	Index name; current default `fde-it-support-nomic-768`.
`pinecone_namespace`	Logical isolation; default `tunisair-fr`.
`ollama_base_url`, `ollama_chat_model`, `embedding_model`	Defaults: host port 11434, `llama3.2`, `nomic-embed-text`.
`top_k`, `max_retries`	Default four retrieved chunks and one bounded rewrite retry.
`audit_db_path`, `tunisair_content_db_path`	SQLite file locations under `data/`.
`tunisair_crawl_delay_seconds`, `tunisair_request_timeout_seconds`	Fetch pacing and timeout defaults: 1 second and 20 seconds.

Settings load from the project-root `.env` and environment variables; never include credential values in a presentation. Some variables from previous experiments may remain in `.env` but are not consumed by the current Ollama-based workflow.

Other repository files

File/folder	Use
`requirements.txt`	FastAPI/Uvicorn, LangGraph/LangChain, Ollama, Pinecone and document loader dependencies.
`Dockerfile`	Builds the Python web app image and starts Uvicorn on port 8080.
`docker-compose.yml`	Runs Ollama and persists model files in a Docker volume.
`.gitignore` / `.dockerignore`	Exclude secrets, local databases/logs and build-only files as configured.
`create_project.py`	Basic scaffold helper that creates directories and empty starter files; not required to run an existing app.
`README.md` / `steps.md`	Project notes and setup guidance; this guide explains the implementation in more depth.
`Agentic-RAG-demo/`	Separate legacy/demo notebook and requirements; not part of the current FastAPI runtime path.
`data/sample_kb/`	Older sample IT handbook/runbook materials; the current web sync indexes the fixed flight-source allowlist instead.
`docs/*.pdf`	Pre-existing FDE project reference material, separate from the new developer guide.

13 · RUNBOOK

Run and update the project

One-time prerequisites

1. Use the project Python virtual environment and install `requirements.txt`.
2. Start Docker Desktop and Ollama: `docker compose up -d ollama`.
3. Ensure Ollama has `llama3.2` and `nomic-embed-text`.
4. Configure a valid Pinecone API key and a 768-dimensional compatible index in private environment settings.

Safe initial/update sequence (PowerShell at repository root)

```
docker compose up -d ollama
Invoke-RestMethod http://localhost:11434/api/tags
.\venv\Scripts\python.exe sync_tunisair.py --dry-run
.\run_weekly_sync.ps1
Get-Content .\data\tunisair_sync.log -Tail 20
```

The dry run reports the four allowed URLs and chunk count; it does not modify SQLite or Pinecone. The runner performs the update and writes a persistent log.

Start the website (separate terminal)

```
.\venv\Scripts\python.exe run.py
```

Open http://127.0.0.1:8080. Check service health at http://127.0.0.1:8080/api/health. Stop the development server with Ctrl+C.

Manual sync versus scheduled sync

Manual update: `.\run_weekly_sync.ps1`. Register/re-register schedule: `.\setup_weekly_sync.ps1`. The scheduled task runs Sunday at 03:00 and assumes Docker/Ollama and network are available then. Inspect task status using `Get-ScheduledTaskInfo -TaskName "YourFlightSupportBot-TunisairSync"`.

> If a source blocks the crawler or shows CAPTCHA, stop and inspect the logs. The correct action is not to automate retries or CAPTCHA solving.

14 · VERIFICATION

Testing and demo plan

Automated regression suite

```
.\venv\Scripts\python.exe -m unittest discover -s tests -v
```

The current suite covers: exact four-page allowlist; no arbitrary route fetch; robots/access-block handling; no redirects; booking URL validation; deterministic route/date extraction; official links for offers and live flight status; structured output mode; SQLite persistence; unchanged-page sync; blocked fetch safety; partial archive when vector sync is unavailable; and Pinecone dimension mismatch safety.

Latest verified repository state before guide generation: **19 tests passed**. The dry run successfully read four public pages and counted 33 chunks. A full update was also completed: 4 pages crawled/stored; 4 pages indexed; 33 chunks upserted. A later unchanged sync returned 0 re-indexed pages and 4 unchanged.

Suggested live demo

1. Show the architecture and explain why the crawler is allowlist-only.
2. Run a dry-run and show allowed URLs/chunks without writes.
3. Ask a content question about offers; open the citation to verify the source.
4. Ask “Donne-moi le lien des offres” and show the official offers URL.
5. Ask “Quel est le statut des vols en cours ?” and explain why live status is linked directly instead of trusting a weekly snapshot.
6. Ask “Quel est le prix du vol Tunis-Montréal ?” and explain dates/availability determine the fare; show the official flight planner.
7. Run the test suite and show the sync log/Task Scheduler status.

Example API test

```
$body = @{ question = "Donne-moi le lien des offres" } | ConvertTo-Json
Invoke-RestMethod -Uri "http://127.0.0.1:8080/api/chat" `
  -Method Post -ContentType "application/json" -Body $body |
  ConvertTo-Json -Depth 8
```

15 · LIMITATIONS

Known limitations and responsible operation

- **Four pages only:** detail pages under offers or destinations are not crawled unless separately reviewed and explicitly allowlisted.
- **HTML text extraction:** no JavaScript execution, browser rendering, PDF downloads or linked page discovery. Dynamic content may be absent from the source HTML.
- **Weekly snapshots:** not suitable as a source for live flight status, fare or seat availability.
- **Local archive:** SQLite data is on the developer machine unless a deliberate backup strategy is added. The database is ignored by Git and must be backed up separately if retention matters.
- **External services:** chat/embeddings require running Ollama; semantic retrieval and indexing require Pinecone connectivity and a valid key.
- **Airport aliases:** only explicitly coded city aliases are recognized. Expand with tests; do not rely on the LLM to guess a code.
- **Audit privacy:** audit records contain raw user questions. Set retention and access controls appropriate to the deployment.
- **Task Scheduler:** this workstation task is configured for the interactive user. A production deployment should use a managed scheduler/identity and explicit monitoring.

Operational failure signals

Signal	Meaning / response
“Tunisair denied automated access” / challenge	Stop sync; do not retry repeatedly or circumvent access controls.
Ollama connection error	Check Docker Desktop/container status and host port 11434; then rerun the sync.
Pinecone dimension mismatch	Configure a compatible 768-dimension index; the code deliberately does not delete an existing index.
Pages unchanged	Expected: skip unnecessary vector writes; check SQLite fetch timestamps if necessary.
Weak evidence	Expected safe behavior: bounded rewrite, then an insufficient-evidence response.

Developer presentation narrative

Short opening (about one minute)

"I built a flight-support assistant that combines a web interface with a controlled RAG pipeline. I chose a fixed list of public airline pages so the system can ground answers in official material instead of searching arbitrary URLs. Extracted source text is archived in SQLite; searchable chunks are embedded locally with Ollama and indexed in Pinecone. LangGraph routes each question, evaluates retrieved evidence and keeps retries bounded. For information that changes quickly—fares and flight status—the assistant links to the airline's live page rather than guessing from an old snapshot."

Questions an evaluator may ask

Question	Short, accurate answer
Why use both SQLite and Pinecone?	SQLite preserves the complete extracted text and sync metadata; Pinecone provides semantic vector search over smaller chunks.
Why not crawl every link?	A fixed allowlist keeps collection predictable, minimizes unnecessary requests and respects site access policy.
How do you avoid hallucinated fares?	Fare queries are routed to flight planning; current prices are not inferred from weekly RAG data.
What makes this agentic?	The graph selects paths, grades evidence, conditionally rewrites and retries once, and stops when evidence remains weak.
How are updates performed?	Manual PowerShell runner or scheduled weekly task; hashes skip unchanged pages and stable vector IDs support replacement.
What is the main improvement area?	With permission, expand curated sources and parser coverage, then improve freshness monitoring, logging and production deployment.

Glossary

RAG: retrieval-augmented generation. **Chunk:** overlapping piece of page text used for search. **Embedding:** numeric representation used to compare text meaning. **Vector store:** database used for similarity retrieval. **Namespace:** logical partition inside Pinecone. **Allowlist:** explicit set of URLs the crawler may request. **Manifest:** local state mapping page hashes to vector IDs.

Guide prepared from the project source and observed sync/test results on 11 November 2026. Verify current source behavior before presenting any live fare, status or service claim.

APPENDIX · FILE AND TEST INDEX

Complete file inventory

Project files by area

Area	Files
Web application	`app/main.py`; `app/api/routes.py`; `app/core/config.py`; `app/core/logging.py`; package `__init__.py` files.
RAG	`app/rag/state.py`; `app/rag/vectorstore.py`; `app/rag/workflow.py`.
Services	`app/services/audit.py`; `content_store.py`; `ingestion.py`; `tunisair.py`; `tunisair_sync.py`.
Run/sync	`run.py`; `sync_tunisair.py`; `run_weekly_sync.ps1`; `setup_weekly_sync.ps1`.
UI	`templates/index.html`; `static/css/style.css`; `static/js/app.js`.
Test/config/docs	`tests/test_tunisair.py`; `requirements.txt`; `Dockerfile`; `docker-compose.yml`; `.gitignore`; `.dockerignore`; `README.md`; `steps.md`; `create_project.py`.
Adjacent legacy/reference material	`Agentic-RAG-demo/`; `data/sample_kb/`; prior PDFs in `docs/`; `LICENSE`. These are not required for the current runtime path.

Regression tests by group

- **Allowlist/crawler:** exact four URLs, block unknown routes, stop on denied access, reject redirects.
- **Booking:** construct a round-trip URL and reject invalid codes/date ordering.
- **Workflow:** return links for offers/status; extract Tunis–Montréal and explicit dates; use compatible structured output; direct fare query safely to the official planner.
- **Sync/storage:** unchanged pages do not re-index; blocked pages do not mutate data; SQLite archive survives embedding failure.
- **Vector safety:** dimension mismatch raises without deleting the existing index.

This inventory summarizes responsibilities; source code remains the authority for exact implementation details. The current unit tests do not require a live website or Pinecone account, except for separate manual integration checks.

APPENDIX · TEST FUNCTION REFERENCE

Automated test cases

Every test below is defined in `tests/test_tunisair.py`. Tests use mocks or temporary SQLite files so the unit suite does not crawl the live airline site or write to the real Pinecone index.

Test method	Behavior protected
`test_only_the_four_explicit_landing_pages_are_fetched`	Only the four fixed approved URLs are requested.
`test_crawler_rejects_non_allowlisted_routes_without_requesting_them`	Unapproved nested routes fail before network access.
`test_challenge_or_forbidden_page_stops_without_retrying`	HTTP 403 becomes an explicit stop; the page request is not retried.
`test_redirect_handler_does_not_follow_redirects`	Redirects are not followed to other pages or hosts.
`test_booking_link_uses_validated_codes_and_dates`	Known valid values map to the expected booking URL parameters.
`test_booking_link_rejects_bad_codes_or_date_order`	Malformed codes and return-before-departure dates are rejected.
`test_booking_route_generates_a_link_without_fetching_booking_host`	A timetable URL is generated without fetching the booking site.
`test_user_link_request_returns_the_offers_page_url`	An offers-link question returns the fixed offers URL.
`test_live_flight_status_uses_current_page_not_weekly_indexed_data`	Live status requests link directly to the current source page.
`test_fare_question_routes_to_booking_and_extracts_tunis_montreal`	Fare intent maps Tunis–Montréal to TUN–YUL without requiring dates.
`test_flight_search_extracts_explicit_french_dates_in_order`	French named-month dates are parsed in trip order.
`test_incomplete_fare_search_links_to_official_flight_planner`	Missing dates prompt the user and include the official planning page.
`test_french_fare_question_returns_planner_link_without_retrieval`	The original French fare query bypasses KB and follows the safe fare path.
`test_router_uses_function_calling_for_route_decision`	Router requests the local model's supported structured-output method.
`test_evidence_grader_uses_function_calling`	Evidence grading uses compatible typed output.
`test_sync_uses_manifest_ids_and_ignores_unchanged_pages`	Stable IDs are used and unchanged text does not cause vector writes.
`test_blocked_fetch_does_not_change_vectors_or_manifest`	Access denial does not update Pinecone or the manifest.
`test_extracted_text_is_archived_when_vector_sync_is_unavailable`	Successfully fetched source text is archived even if embedding is offline; manifest remains pending.
`test_dimension_mismatch_does_not_delete_existing_index`	A dimension mismatch errors safely without deleting the existing Pinecone index.

> **Latest verified result:** 19 tests passed using `python -m unittest discover -s tests -v`.